INTRODUCCIÓ ALS ORDINADORS

INFORME PRÀCTICA 4

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

INTRODUCCIÓ

En aquesta pràctica, s'observarà com funciona el programa Ripes utilitzant el processador Pipeline de 5 etapes, i per tant també es veurà com funciona el programa i quin és l'estat de les etapes del Pipeline en cada moment. A més a més, s'incentivarà l'ús de les Pipeline tables per tal de veure la taula d'estats de les etapes en cada cicle del programa. Es veuran situacions com FLUSH i STALL durant l'execució dels codis proposats i també s'analitzaran les sortides dels diferents components del circuit que ens mostra el simulador RIPES, on es veuen les 5 etapes. Per últim, també es veurà a quines posicions de memòria el PC anirà en funció del que passi en les diferents etapes del Pipeline.

OBJECTIUS

L'objectiu d'aquesta pràctica es fer un ús més extens del programa RIPES, on s'aprendrà a entendre el funcionament de les Pipeline tables i com es relaciona amb l'esquema de les etapes del Pipeline de Ripes. A més a més, també es té com a objectiu aprendre les diferents etapes del Pipeline i veure en quins moments es produeixen situacions com FLUSH i STALL, a més a més de comprendre les sortides dels diferents components del circuit de Ripes i veure com funciona el PC en aquest processador. Amb aquesta pràctica, també es vol aprendre a identificar el nombre de cicles o el comportament de codis que fan el mateix però que són diferents.

Exercici guiat

Haurem d'executar els següents codis, cicle a cicle (microinstrucció a microinstrucció) utilitzant com a micro el 5-Stage Processor al simulador Ripes:

Codi 1

```
.data
resultat: .word 0
.text
main:
add a3, zero, zero
add a7, zero, zero
addi a2, zero, 4
add a3, a2, a3
addi a7, a7, -1
la a0, resultat
sw a3, 0(a0)
```

Codi 2

```
.data
resultat: .word 0
.text
main:
add a3, zero, zero
add a7, zero, zero
addi a2, zero, 4
add a3, a2, a3
addi a7, a7, -1
blt a7, zero, salta
salta:
la a0, resultat
sw a3, 0(a0)
```

1) Abans d'executar els codis tracta d'esbrinar la seva funcionalitat. Faran el mateix? Creus què trigaran el mateix nombre de cicles en executar-se?

La seva funcionalitat serà la mateixa en els dos codis, ja que només difereixen en un salt condicional, que es donarà sempre, ja que el registre a7 inicialitza el seu contingut en 0 amb la instrucció ADD a7, zero, zero (es fa 0+0=0 i es guarda el resultat en a7) i després se li resta l'immediat -1 amb la instrucció ADDI a7, a7, -1, que guardarà en el registre a7 el -1. Seguidament, en el cas del codi 1, és carrega el valor de "resultat" en el registre a0 amb la instrucció LA a0, resultat i després es guarda el valor de a3 en la posició de memòria 0 + [a0] amb la instrucció SW a3, 0(a0)

En el cas del codi 2 es fa ús de la instrucció BLT a7, zero, salta , que sempre serà cert degut al funcionament del codi. En aquest cas, es farà un salt de posició de memòria a l'etiqueta "salta" i es faran les instruccions anteriors.

Per tant, observem que els dos codis realitzaran la mateixa funció. No obstant, sí que és cert que no trigaran el mateix nombre de cicles, ja que en el cas del codi 2, el que haurà de fer és fer el càlcul de la comparació i a més a més, calcular on ha de fer el salt de memòria, ja que es suma el offset (en aquest cas "salta") amb el PC.

2) Ves a la finestra del Ripes dedicada al processador (Processor). Busca entre les opcions del Simulator control, la Pipeline table.

La Pipeline table ens servirà per veure com s'estan executant les instruccions segons el cicle on estigui el programa.

3) Executa els dos codis cicle a cicle fixant-te com les instruccions van passant per les diferents etapes del pipeline. Compta els nombre de cicles que es necessiten per executar cadascun dels codis i compara les Pipeline tables.

Observem que les dades que hi ha a la finestra del processador ens indiquen que en el cas del codi 1 es fan 12 cicles per realitzar la seva execució, mentre que en el cas del codi 2 es fan 15 cicles.
Veiem a continuació les Pipeline tables dels dos codis:

Codi 1

	0	1	2	3	4	5	6	7	8	9	10	11	12
add x13 x0 x0	IF	ID	EX	MEM	WB								
add x17 x0 x0		IF	ID	EX	MEM	WB							
addi x12 x0 4			IF	ID	EX	MEM	WB						
add x13 x12 x13				IF	ID	EX	MEM	WB					
addi x17 x17 -1					IF	ID	EX	MEM	WB				
auipc x10 0x10000						IF	ID	EX	MEM	WB			
addi x10 x10 -20							IF	ID	EX	MEM	WB		
sw x13 0 x10								IF	ID	EX	MEM	WB	

Codi 2

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
add x13 x0 x0	IF	ID	EX	MEM	WB											
add x17 x0 x0		IF	ID	EX	MEM	WB										
addi x12 x0 4			IF	ID	EX	MEM	WB									
add x13 x12 x13				IF	ID	EX	MEM	WB								
addi x17 x17 -1					IF	ID	EX	MEM	WB							
blt x17 x0 4 <salta>						IF	ID	EX	MEM	WB						
auipc x10 0x10000							IF	ID	IF	ID	EX	MEM	WB			
addi x10 x10 -24								IF		IF	ID	EX	MEM	WB		
sw x13 0 x10											IF	ID	EX	MEM	WB	

Observem que en el primer codi, les cinc etapes en una instrucció es desenvolupen en l'ordre establert, és a dir, per cada cicle la instrucció passa per una etapa diferent, i la instrucció següent segueix amb l'etapa de la instrucció anterior, així fins realitzar el WriteBack (WB) en el cicle corresponent. En canvi, en el codi 2, s'observa que fins la instrucció de BLT, el programa funciona de la mateixa forma que el del codi 1. No obstant, en el moment de realitzar la instrucció BLT, el primer que es fa és el Instruction Fetch (IF), per tant, es llegeix la següent instrucció obligatòriament, per això apareix el IF a la següent instrucció en el següent cicle. Una vegada es fa el IF, es passa al Instruction Decode (ID), en el que es llegeixen els registres d'operands, i a més a més també es computen les direccions de salt, tal i com passa en aquest cas amb la instrucció BLT. Per tant, en el moment en que s'executa la instrucció, es veurà que al cicle següent, la instrucció auipc tornarà a l'etapa de IF. A partir d'aquest moment, el programa del codi 2 torna a comportar-se com el codi 1.

4) Per què les etapes de la instrucció auipc x10 0x 10000 al pipeline son IF ID IF ID EX MEM WB al Codi 2?

Perquè el processador fa una predicció i suposa que la instrucció de salt (en aquest cas tenim BLT a7, zero, salta) és certa, és a dir que es compleix que el contingut del registre a7 és menor estricte a zero (a7<0). Per tant, fins que no s'executa (EX) la instrucció BLT, el processador segueix amb les microinstruccions que són de la predicció (FLUSH), tal que les primeres etapes de auipc x10 0x 10000 són IF ID. En el

moment que s'executa BLT, es produeix en aquest programa el salt, i llavors en el següent cicle, es torna al IF en la instrucció auipc x10 0x 10000 , ja que la predicció que havia fet el programa era falsa, i per tant es va a la posició de memòria corresponent a la que es marca al salt, que ha estat prèviament calculada a l'etapa ID de la instrucció BLT. Per tant, a partir de llavors, el programa continua com el del codi 1, on auipc x10 0x 10000 seguirà amb les etapes IF ID EX MEM WB. Per tant, veiem que per això la instrucció auipc x10 0x 10000 té com a etapes IF ID IF ID EX MEM WB, etapes pròpies del pipeline.

5) Com afectaria al nombre total de cicles d'execució el següent canvi en el codi:

Codi 1

```
.data
resultat: .word 0
.text
main:
add a3, zero, zero
add a7, zero, zero
addi a2, zero, 4
add a3, a2, a3
addi a7, a7, -1
la a0, resultat
sw a3, 0(a0)
```

---------->

Codi 3

```
.data
resultat: .word 0
.text
main:
add a3, zero, zero
add a7, zero, zero
addi a2, zero, 4
add a3, a2, a3
addi a7, a7, -1
bgt a7, zero, salta
salta:
la a0, resultat
sw a3, 0(a0)
```

Observem que en la finestra del processador, en el cas del codi 1 es triga 12 cicles en realitzar el programa, mentre que el canvi que proposa l'exercici fa que trigui 13 cicles. Això es deu a tenir una instrucció més que el codi 1, ja que en aquest cas, la predicció del processador fa que, com la condició de BGT a7, zero, salta és falsa, llavors es continua amb les etapes corresponents. D'aquesta forma, quan la instrucció de salt arriba a la seva etapa d'execució, com no hi ha cap salt a realitzar perquè en ID no s'ha

calculat cap salt perquè la condició és falsa, el processador no esborra el FLUSH i per tant es segueix des de la posició de memòria on es troba el PC en aquell moment, és a dir, el programa segueix d'acord amb la seva predicció. Per tant, cada instrucció tindrà com a etapes ID IF EX MEM WB, i per tant només es tindrà incrementat el nombre de cicles en 1 pel nombre d'instruccions de més.

PRÀCTICA A REALITZAR

Ripes: Executa els següents programes microinstrucció a microinstrucció:

Codi 4

```
.data
valorDada: .word 2
guardaResultat: .word 0
.text
main:
la a0, valorDada
lw a7, 0(a0)
addi a2, zero, 9
add a3, zero, zero
loop:
add a3, a2, a3
addi a7, a7, -1
bgt a7, zero, loop
la a0, guardaResultat
sw a3, 0(a0)
```

Codi 5

```
.data
valorDada: .word 2
guardaResultat: .word 0
.text
main:
la a0, valorDada
lw a7, 0(a0)
addi a2, zero, 9
add a3, zero, zero
loop:
add a3, a2, a3
addi a7, a7, -1
bgt a7, zero, loop
la a0, guardaResultat
sw a3, 0(a0)
lw a0, guardaResultat
addi a0,a0,4
```

En el codi de l'exercici 5, es produeix un retard d'execució (STALL) ja que per poder executar la instrucció addi a0, a0, 4 necessitem actualitzar el valor de a0 definit en el WB de la instrucció anterior. Explica amb les teves paraules què és un STALL

Un STALL és una situació en la que el processador es pausa temporalment per falta de dades o d'instruccions. Quan això passa, el processador espera a que arribin les dades o es realitzin les instruccions necessàries per tal de poder seguir amb el processament. En aquest cas, com estem utilitzant el Pipeline de 5 etapes com a processador, el que passa és que com es fa un load word (LW) en a0 i seguidament la instrucció ADDI a0,a0,4 , llavors la instrucció ADDI ha d'esperar al writeback (WB) de la instrucció LW a0, guardaResultat per tal de poder fer l'execució de la instrucció de ADDI a0, a0, 4. Llavors aquest és el motiu de la situació de STALL.

Per resoldre aquestes qüestions és necessari mirar l'estat del pipeline o utilitzar la Pipeline table (feu servir els codis 3 i 5 per respondre a aquestes qüestions):

1) Quin es l'estat de cadascuna de les cinc etapes del pipeline al cicle 6? I al 8?

A les taules del Pipeline s'observa que al cicle 6, veiem que l'estat de cadascuna de les etapes en el codi 5 és el següent:
ID: addi x17 x17 -1 IF: add x13 x12 x13 EX: add x13 x0 x0 MEM: addi x12 x0 9 WB: lw x17 0 x10
En el cicle 6, els estats de les etapes del Pipeline en el codi 5 són les següents:
ID: auipc x10 0x10000 IF: sw x13 0 x10 EX: addi x17 x17 -1 MEM: add x13 x12 x13 WB: add x13 x0 x0

Per altra banda, en el cas del codi 3, els estats de les etapes en el cicle 6 són les següents:
ID: auipc x10 0x10000 IF: blt x0 x17 4 <salta> EX: addi x17 x17 -1 MEM: add x13 x12 x13 WB: addi x12 x0 4
Mentre que al cicle 8 són els següents:
ID: sw x13 0 x10 IF: addi x10 x10 -24 EX: auipc x10 0x10000
MEM: blt x0 x17 4 <salta> WB: addi x17 x17 -1

2) Quins senyals de control s'activen en el cicle 4 a la segona etapa del pipeline? A quines instruccions del codi corresponen?

Tant en el codi 3 com en el codi 5 s'observa que en el cicle 4 a la segona etapa del pipeline s'activa el Write Enable dels Registres. En particular, les senyals de control corresponen a la instrucció ADD x13 x12 x13 , que correspon a la instrucció de ADD. a3. a2, a3 dels codis 3 i 5.

3) Quins són els valors a les sortides dels multiplexors assenyalats a la figura al cicle 9?

Observem que en els dos multiplexors assenyalats tenim com a sortides, en valor hexadecimal, 0x10000018 en el multiplexor superior i 0xffffffe8 en el multiplexor inferior, d'acord amb la imatge en el codi 3. En el codi 5, els valors són 0x0000001c en el multiplexor superior i 0xffffffff8 en el multiplexor inferior.

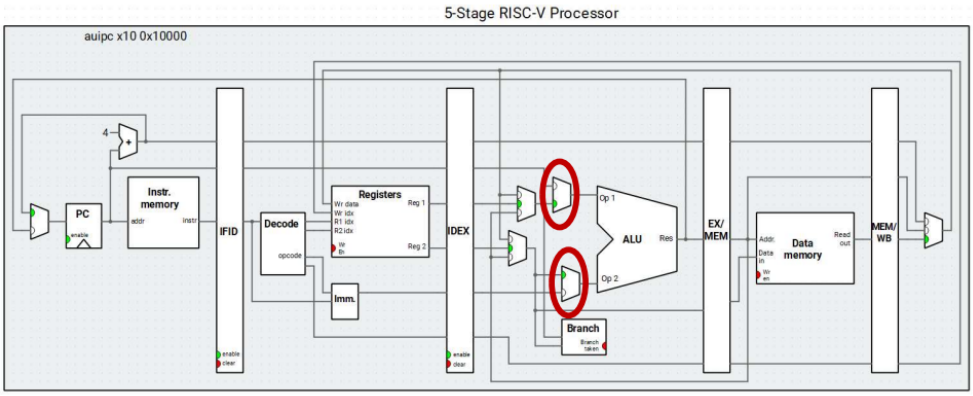

Figura 4. Detall de les etapes del RISC-V en el simudor RIPES.

4) Què està calculant la ALU al cicle 9?

Veiem que en el codi 5, l'ALU està realitzant una suma dels valors que hem esmentat prèviament. En particular, el que està fent és calcular la posició de memòria on el PC es situarà en el cas que la condició es compleixi, és a dir al PC se li està sumant un offset negatiu en aquest cas, tal que si es compleix el PC anirà a una posició anterior. Això s'està realitzant amb la instrucció a codi màquina blt x0 x17 -8 <loop>, és a dir la instrucció BGT a7, zero, loop.

En el codi 3, també està fent la suma amb els valor esmentats a la pregunta anterior i dóna com a resultat 0x10000000. En particular, el que està fent és donar el valor al global pointer, per tal de poder fer el Load Adress amb el registre a0 i el contingut de "resultat", és a dir la instrucció LA a0, resultat.

5) Llegeix amb cura la part del codi amb que s'implementa el loop. Tenint en compte el que has vist a la qüestió 3), justifica perquè el pipeline al cicle 10 presenta aquest estat:

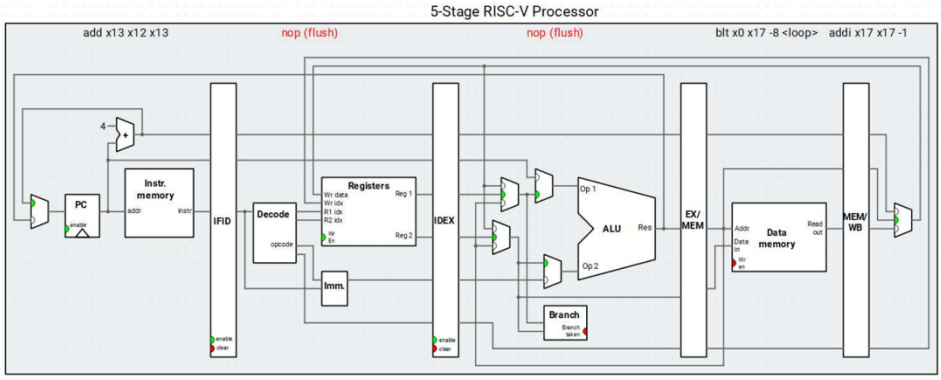

Figura 5. Estat del pipeline al cicle 10.
Les el·lipses vermelles assenyalen els multiplexors que s'utilitzen a la qüestió 3.

Observem el codi 5 amb atenció i veiem que en la primera iteració del loop es compleix la condició de que el contingut del registre a7 sigui més gran estricte a 0 (a7>0), per tant va a la posició de memòria corresponent a "loop". Aquí hem de parar atenció ja que el que succeeix és, com ja hem vist abans, que les prediccions que fa el pipeline en aquest cas són falses, i per tant ha d'esborrar aquestes etapes en el següent cicle. Llavors, en el cicle 10 l'etapa IF correspondrà a la primera línia de codi que hi ha després de "loop". En canvi, les etapes ID i EX, com prèviament en el cicle 9 les etapes IF i ID corresponien a prediccions del pipeline, s'han esborrat perquè eran falses (FLUSH). En particular, a la taula del Pipeline s'observa que al cicle 10 només hi ha les etapes IF MEM WB, donant constància al fet que succeeix a la figura de dalt.

6) Quan NO es produeix el salt, quantes etapes de la instrucció auipc x10 0x 10000 s'executen al pipeline?

Tenint en compte que estem parlant del codi 5, observem que si no es produeix el salt de posició de memòria que pot provocar la instrucció BGT a7, zero, loop, aleshores s'executen les 5 etapes del pipeline, que són ID IF EX MEM WB. Per tant, s'executarien 5 etapes.

7) Quan s’està executant la instrucció de salt (etapa EX), però el salt NO es produeix, a quina posició apunta el program counter (PC)?

Si el salt no es produeix, aleshores les prediccions del Pipeline seran correctes, i per tant en el moment en el que la instrucció de salt està en la etapa EX, el PC està apuntant a la posició de memòria 0x00000024. En el següent cicle, on la instrucció de salt estarà en l'etapa MEM, aleshores el PC es pot comprovar que apunta a la posició 0x00000028.

8) Quan es produeix el salt, quantes etapes de la instrucció auipc x10 0x 10000 s’executen al pipeline?

En el cas de produir-se el salt, s'executen només 2 etapes, que són ID IF, ja que en el següent cicle, la instrucció de salt ja s'haurà executat, i per tant estarà en l'etapa MEM, amb la qual cosa l'etapa IF d'aquell cicle del pipeline correspondrà a la instrucció de la posició de memòria a la que s'ha saltat i ja no es produeix l'etapa EX en la instrucció auipc x10 0x10000.

9) Quan s’està executant la instrucción de salt (etapa EX), i el salt es produeix, a quina posició apunta el program counter (PC)?

Si el salt es produeix, aleshores les prediccions del Pipeline seran incorrectes, i per tant en el moment en el que la instrucció de salt està en la etapa EX, el PC està apuntant a la posició de memòria 0x00000024. En el següent cicle, on la instrucció de salt estarà en l'etapa MEM, aleshores el PC es pot comprovar que apunta a la posició 0x00000014, que és correcte, ja que s'ha produït un salt cap enrere i veiem que aquesta posició de memòria és anterior a la que estava abans.

CONCLUSIONS

Com a conclusions d'aquesta pràctica, veiem que s'ha treballat en saber comprendre què és el que està succeint durant l'execució d'un codi, mitjançant l'ús de les Pipeline tables o l'observació dels estats de les 5 etapes del Pipeline. A més a més, s'ha comprovat que malgrat dos codis facin la mateixa funció, poden tenir diferent nombre de cicles i diferents comportaments en els estats de les etapes del Pipeline.

Per altra banda, també hem pogut observar que les instruccions de salt provoquen que les instruccions que hi ha després no facin les 5 etapes del Pipeline, ja que en produir-se el salt de memòria, el PC deixa d'apuntar on estava apuntant i els estats de les etapes del Pipeline són uns altres diferents als anteriors, el que s'anomena FLUSH.

A més a més, també s'ha treballat en la interpretació de les sortides de diferents components com els multiplexors o l'ALU, en la que a partir d'aquestes, es podia comprendre què estava succeint en el programa, com per exemple diferents situacions com FLUSH o STALL.

Per tant, aquesta pràctica ha servit per aprendre a fer un ús més avançat i analític del programa Ripes, fent ús de les diferents eines que ens ofereix per tal de comprendre el funcionament dels diferents codis, com per exemple saber el nombre de cicles i els estats de les diferents etapes del Pipeline.